Ders 2 Çalışma Özeti

Ders 2 Çalışma Özeti

Genel Konular

- Tasarımın öğrenilmesi ve dersin yöntemi
  - Mimari bir yapı kurmak için önce yazılımın kendisi düzgün tasarlanmış olmalıdır. Mimari kadar tasarım da doğru yapılmalıdır.
  - Tasarımın "şunu şununla topla, sonra şununla çarp" gibi net bir tarifi yoktur. Hoca tavsiye verebilir; tasarım öğretilmekten çok yaparak öğrenilir.
- Örnek problem: basit bir metin görüntüleyici (text view)
  - Diskteki bir metin dosyasını açıp satır satır ekranda gösteren basit bir araç tasarlanır. Hazır bir "dosya görüntüleme" bileşeni yoktur; çözüm bileşenlerden kurulacaktır.
  - İlk soru (ekrandan, 00:17:15): Swing ya da SWT gibi bir GUI kütüphanesinde Checkbox, Window, Progress Bar, Combo Box, Scroll Bar ve Radio Button varken hangileri gerekir? Cevap: bir pencere (Window) ve bir kaydırma çubuğu (Scroll Bar).
  - Metni GUI sağlamaz, bu yüzden üçüncü bir bileşen gerekir: DosyaYöneticisi (ekrandan, 00:20:00).
- DosyaYöneticisi kısıtları (ekrandan, 00:20:00)
  - Dosyanın tüm içeriği hafızada tutulamaz (ör. 4 GB'lık dosya, 2 GB belleği olan sanal makine). Kalanı için diske erişip okuma yapmak gerekir.
  - İşletim sistemi seviyesinde dosyaya satır satır erişim vardır. Kütüphanedeki satır okuma modülü istenen uzunluğa kadar ardışık satırları verir, dosya açma ve kapatma ayrıntılarını halleder.
  - Okuma sorumluluğu uygulamanın içine gömülmez, ayrı bir bileşene hizmet olarak verilir.
- Gösterici (pencere) gereksinimleri (ekrandan, 00:40:00)
  - Tamsayı (integer) satır sayısı gösterir; boyutu 1 ile 100 satır arasında değiştirilebilir.
  - Yazı tipi ve punto sabittir (terminaldeki vi gibi). Değişseydi satır sayısını hesaplayan ara bir bileşen gerekirdi. Yatay kaydırma da kapsam dışı bırakılır.
- Kaydırma çubuğu (ekrandan, 00:30:00)
  - Hareket eden parçaya "kulp", kulbun hareket ettiği alana "tabla" denir.
  - Kulp en üstteyken dosyanın başı, en alttayken sonu gösterilir.
  - Kulbun boyutunun tabla boyutuna oranı, dosyanın gösterilen parçasının oranına eşittir. Tüm dosya sığıyorsa kulp tüm tablayı kaplar; dosya çok büyükse ya da gösterici çok küçükse incecik bir kulp görülür.
- Kullanım senaryoları ve analiz modeli
  - Üç senaryo: metin görüntüleme, kaydırma çubuğunu hareket ettirme, göstericiyi yeniden boyutlandırma (ekrandan, 00:31:35).
  - Senaryolar sistemin kullanıcıya vereceği tepkileri ortaya koyar; bunlar use case modellemesiyle ifade edilir. Bileşenler, davranışlar ve özellikler sınıf diyagramıyla gösterildiğinde analiz modeli elde edilir.
- UML sınıf diyagramı
  - Sınıf kutusu üç bölmelidir: ad, özellikler, işlemler. Daha fazla bölme de olabilir.
  - Analizde bulunan sınıflar kavramsal (aday) sınıflardır; gerçekleştirme ayrıntısına girilmez.
  - İşlemler, sistemin kullanıcıyla etkileşimini sağlayan davranışlardır: `boyutlandır(yeniBoyut)`, `kaydır(yeniKonum)` (ekrandan, 00:39:50).
  - Yalnız temel UML veri tipleri (string, integer, void) kullanılır. Etkisi için çağrılan işlemin dönüş tipi `void`'dir.

- Görünen özellikler (percept) (ekrandan, 00:46:30)
  - Kullanıcının görebildiği özelliklere algı (percept) denir: kulbun pozisyonu, göstericinin boyutları, kulbun boyutu, pencerenin içindeki metin.
  - DosyaYöneticisi kullanıcıyla doğrudan etkileşmez; onun görünen özelliği işletim sisteminden aldığı belgedir.
  - Kullanıcı ve dosya sistemi (işletim sistemi) dış paydaştır; bu soru sistemin aktörlerini bulmayı kolaylaştırır.
- İlişkiler (ekrandan, 01:03:00)
  - Sınıflar ve özellikler kolay kısımdır; zor olan bileşenler arası ilişkileri bulmaktır.
  - UML analiz modelinde üç ilişki türü: işbirliği (association), birleşme (aggregation), genelleme (generalization).
  - İlişkiler, senaryolarda hangi bileşenin hangi kullanıcı eylemine karşılık vermekten sorumlu olduğu sorulardan çıkarılır.
- Görünen satır sayısı ilişkisi (ekrandan, 01:08:15 ve 01:10:00)
  - görünenSatırSayısı $= \min(\text{Pencere Boyutu}, \text{Dosya Satır Sayısı})$.
  - Sınıflar: KaydırmaÇubuğu (`kulpBoyutu : Integer`, `kulpKonumu : Integer`, `kaydır(yeniKonum : Integer)`), DosyaYöneticisi (`belge : dizi(satır)`), Gösterici (`yükseklik : Integer`, `içerik : dizi(satır)`, `boyutlandır(yeniBoyut : Integer)`); DosyaYöneticisi ile Gösterici arasında `GörünenSatırlar` ilişkisi.
- Hangi satırlar görünecek (ekrandan, 01:12:20)
  - Görünecek en üst satır + görünen satır sayısı. Dayanak: Gösterici boyutu, DosyaYöneticisi belge boyutu, KaydırmaÇubuğu kulp konumu. Bu üç bileşen arasında üçlü (ternary) bir ilişkidir.
- Kulp boyutu (ekrandan, 01:14:30)
  - kulpOranı $=$ GörünenSatırSayısı$/$dosyaBoyutu; oran 1 ise kulp tüm tablayı kaplar.
  - 100 satır görünüyor ve dosyada 1000 satır varsa kulp tablanın %10'unu kaplar. Bu da File Manager, Window ve Scrollbar arasında üçlü bir ilişkidir.
- Genel tasarım kavramları
  - Tasarım her yerdedir: Uluslararası Uzay İstasyonu, planlanan bir aile yemeği (ekrandan, 01:20:00).
  - Tasarım: bilinçli ve amaçlı planlama. Mühendislik: bilimsel ve matematiksel temelleri uygulayan becerili ya da sanatsal üretim. Zanaat: deneyimden gelen yetenekli meslek. Sanat: beceri, tat ve hayal gücünü estetik nesnelerin üretiminde kullanmak.
  - Yazılım tasarımı ile programlamanın farkı: ölçek ve işlevsel olmayan gereksinimlere verilen önem (ekrandan, 01:30:00).
  - Yazılım tasarımı: işlevsel gereksinimleri yerine getirirken işlevsel olmayan gereksinimleri de atlamadan yazılım üretmek.
  - Mimari tasarım: modül ve bileşenlere sorumluluk atamak ve bunların nasıl etkileşeceğini belirlemek. Detaylı tasarım: veri yapıları ve algoritmalarla davranışların içini tanımlamak (sözde kod, yapısal programlama, akış diyagramı, karar tablosu).
- Yazılım tasarımı yaklaşımları (ekrandan, 01:40:00)
  - Nesneye yönelik, yapısal, rol tabanlı; belirli sınıf yazılımlara odaklanan (gerçek zamanlı sistemler); uygulamanın bir kısmına odaklanan (UI/UX).
  - Tüm yaklaşımlarda ortak 3 yön: tasarım yöntemi, tasarım gösterimi, tasarım geçerleme.
- Tasarım ikilemleri (ekrandan, 01:41:00)
  - Yukarıdan aşağı mı, aşağıdan yukarı mı, içten dışa mı; yordam ve fonksiyonlar mı, kavramlar ve davranışlar mı?
  - Tasarım zaman alır: uzun dönem yönetilebilir yapı mı, kısa takvimli işi yetiştirmek mi? Hangi araçlar kullanılacak?
- Geçerleme, belgelendirme ve anahtar kavramlar
  - Geçerleme tasarımcıdan bağımsız bir ekip tarafından yapılmalıdır; hata ne kadar erken bulunursa çözüm o kadar ucuzdur.
  - Geleneksel tasarım belgeleri: bileşenler (sorumlulukları, veri akışları), kontrol akışı, performans kriterleri, kaynak kullanımı. Yetmezse IEEE 1016 standardı (ekrandan, 01:51:10).
  - Anahtar kavramlar: kavramsal bütünlük, az bağımlılık, iyi uyum, bilgi gizleme, soyutlama, estetik (ekrandan, 01:55:50, 01:57:10, 02:00:00).
  - Tasarım felsefesi dört filozofla ilişkilendirilir: Descartes, Marx, Heidegger, Wittgenstein (ekrandan, 02:00:00).

Hocanın Özellikle Vurguladığı Kısımlar

- "Herkes kendi işini yapsın"
  - Her bileşen yalnız uzmanı olduğu bilgiyi tutmalı, fazlasını bilmemelidir. Hocanın sözüyle: ne kadar çok şey bilirseniz o kadar çok şey yapmak zorunda kalırsınız. Bu bir kural değil, tavsiyedir.
- Kaydırma olayının duyurulması
  - `kaydır` bir değer döndürürse diğer bileşenler sürekli kaydırma çubuğuna gidip sormak zorunda kalır, bağımlılık artar.
  - Kaydırma çubuğu "konumum değişti" diye bir olay yayınlarsa ilgilenen bileşenler dinler. Yarın yeni bir bileşen eklendiğinde o da dinlemeye başlar; tasarım daha esnek olur.
  - Kararlarınızı "gelecekte bu bana neye mal olur" diye düşünerek verin.
- Uç durumlar: boş dosya
  - Belgenin satır sayısı sıfırsa kulp boyutu hesabında sıfıra bölünür ve program çöker. Hoca, böyle bir istekte çoğu kişinin sıfır boyut durumunu unutacağını söyler.
- Üçlü ilişkilerden kaçınma
  - Ne kadar çok bağımlı olunursa o kadar değişemez ve yönetilemez hâle gelinir. Mümkünse üçlü ilişkiler ikililere çevrilmelidir. Örnekte pencere ile kaydırma çubuğu ayrılmaz parçalar olduğu için bu sorun göze batmaz.
- Analiz modelinde ayrıntıya girmemek
  - 1–100 aralığı gibi kısıtlar ve "minimum" gibi türetilmiş değerler UML diyagramında gösterilemez; bunun için OCL (Object Constraint Language) kullanılır.
- Moda diye tasarım yapılmaz
  - "Mikroservis çıktı, taşıyalım", "konteyner çıktı, konteynerde çalışalım" demek bilinçli ve amaçlı tasarım değildir. Her yaklaşım belirli problemler içindir; bazen monolitik orta katman daha doğru çözümdür.
- Ödünleşim kaçınılmazdır
  - Performans öncelikli bir hava tahmini uygulamasında dizi, nesne koleksiyonundan hızlıdır. Komşuluk ileride altıgen (bal peteği) olabilecekse dizi kullanılamaz ve performanstan ödün verilir.
- Belgelendirme ve gerekçelendirme
  - Basit notlar ve yorumlar bile değerlidir. Bir yıl sonra koda dönüldüğünde "ben burada ne yapmışım" denir. Tasarım kararlarının gerekçesi mutlaka yazılmalıdır.
- Deneyim
  - Kaliteyi yansıtan en önemli gösterge deneyimdir ("kilometre"). Tasarım yaparak öğrenilir.

Kısa Tekrar Notları

- Metin görüntüleyici bileşenleri: Gösterici (pencere), KaydırmaÇubuğu, DosyaYöneticisi.
- Senaryolar: metin görüntüleme, kaydırma, yeniden boyutlandırma.
- Sınıf kutusu: ad, özellikler, işlemler. Analizde kavramsal sınıflar, temel UML tipleri, etkisi için yapılan işlem `void`.
- Percept: kullanıcının görebildiği özellik.
- Üç ilişki: association, aggregation, generalization. İlişkileri bulmak zor kısımdır.
- Görünen satır sayısı $=\min(\text{pencere boyutu}, \text{dosya satır sayısı})$.
- Görünen satırlar: en üst satır + görünen satır sayısı; dayanak boyut, belge boyutu, kulp konumu (üçlü ilişki).
- Kulp oranı = görünen satır sayısı / dosya boyutu; 100/1000 → %10. Boş dosyada sıfıra bölme tuzağı.
- UML'de gösterilemeyen kısıtlar için OCL.
- Tasarım = bilinçli ve amaçlı planlama. Yazılım tasarımı = işlevsel + işlevsel olmayan gereksinimler arasında denge.
- Tasarım ve programlama farkı: ölçek (ISS'de 30 milyon satır kod) ve işlevsel olmayan gereksinimler.
- Ortak üç yön: yöntem, gösterim, geçerleme. Geçerlemeyi bağımsız biri yapar.
- Belgeleme: IEEE 1016. Kararlar gerekçelendirilir.
- Anahtar kavramlar: kavramsal bütünlük, az bağımlılık, iyi uyum, bilgi gizleme (Parnas), soyutlama, estetik.

- Filozoflar: Descartes (analiz, model), Marx (sosyal etki, kullanıcı merkezli), Heidegger (araç: CASE, IDE), Wittgenstein (dil oyunları, anlamlı adlandırma).

Detaylı Açıklamalar

Dersin ilk yarısı tek bir örnek üzerinden ilerler: diskteki bir metin dosyasını satır satır gösteren basit bir metin görüntüleyici. Hoca, hazır bir dosya gösterme bileşeni olmadığını varsayar ve soruyla başlar: elimizdeki GUI bileşenlerinden hangilerine ihtiyaç var? Checkbox, Progress Bar, Combo Box ve Radio Button gereksizdir. Metin bir pencerede gösterileceği için Window, dosyanın uzunluğu bilinmediği ve ekranda sınırlı satır (örneğin 24) gösterilebildiği için Scroll Bar gerekir. Metne erişimi GUI sağlamadığı için üçüncü bir bileşen, DosyaYöneticisi, eklenir. Hoca bu bileşenleri yalnız düşünerek bulduğunu vurgular: elimde ne var, neye hâlâ ihtiyacım var?

Gereksinimler geldikçe tasarım şekillenir. Dosya belleğe sığmayabilir, bu yüzden DosyaYöneticisi her seferinde gerektiği kadar satırı diskten okumalıdır. İşletim sistemi satır satır erişim sağlar ve kütüphanedeki modül istenen sayıda ardışık satırı verir. Gösterici 1 ile 100 arasında tamsayı satır gösterir. Pencere 20 satırdan 50 satıra büyütülürse eksik 30 satır nasıl istenecek? Baştan 50 satır mı, yoksa yalnız eksik 30 satır mı? Bu kararların hepsi tasarımcınındır. Punto ve yatay kaydırma örneği basit tutmak için kapsam dışı bırakılır. Bunlar istenseydi araya satır sayısını hesaplayan ayrı bir bileşen gerekirdi.

Kaydırma çubuğunda da karar gerektiren durumlar vardır. Kulp en alttayken dosyanın son satırları gösterilmelidir, son satır en tepeye alınmaz. Dosyanın sonundayken pencere 20 satırdan 40 satıra büyütülürse ya son 20 satır ve altında boşluk ya da son 40 satır gösterilir. Hocaya göre ikisi de doğru olabilir, ama biri seçilip davranış ona göre modellenmelidir. Kulbun boyutu da dosya ve pencere boyutundan oransal olarak hesaplanır.

Ardından senaryolar yazılır: metin görüntüleme, kaydırma ve yeniden boyutlandırma. Senaryolar, sistemin kullanıcı gözüyle nasıl kullanılacağını adım adım anlatır ve sorumlulukların bileşenlere atanmasını sağlar. Metni göstermek kullanıcıya bir tepki değildir; kaydırma ve boyutlandırma ise tepki gerektirir. Bu tepkiler sınıf diyagramının üçüncü bölmesine işlem olarak yazılır: `boyutlandır(yeniBoyut : Integer)`, `kaydır(yeniKonum : Integer)`. Boyutlandırma etkisi için yapıldığından dönüş tipi `void`'dir.

`kaydır` işleminin dönüş değeri derste tartışılır. Bir öğrenci kaydırma sonucunda yeni satırların döndürülmesi gerektiğini, konumdan satır aralığının hesaplanması için ara bir katman gerektiğini söyler. Hoca ise kaydırma çubuğunun bir olay yayınlamasını, pencere gibi ilgilenen bileşenlerin bunu dinlemesini önerir. Dönüş değeriyle çözüm bileşenleri kaydırma çubuğuna sürekli sormaya zorlar ve bağımlılığı artırır. Olay yayınında ise yeni eklenen bir bileşen de dinleyici olabilir.

Özellikler de kullanıcının gördüklerinden (percept) çıkarılır: kulbun konumu ve boyutu, göstericinin boyutu ve içindeki metin. DosyaYöneticisinin görünen özelliği, işletim sisteminden aldığı belgedir. Kullanıcı ve işletim sistemi sistemin dışında kalan paydaşlardır.

Molanın ardından ilişkiler ele alınır. Hoca, sınıfları ve özellikleri bulmanın kolay, ilişkileri bulmanın zor olduğunu söyler. Gösterici ile DosyaYöneticisi arasındaki `GörünenSatırlar` ilişkisi, pencere boyutu ve dosya satır sayısının minimumudur. Hangi satırların görüneceği ise en üst satıra görünen satır sayısı eklenerek bulunur. Bunun için Göstericinin boyutu, DosyaYöneticisinin belge boyutu ve KaydırmaÇubuğunun kulp konumu birlikte kullanılır, yani üçlü bir ilişki oluşur. Kulp oranı da görünen satır sayısının dosya boyutuna bölümüdür: 100 satır görünüyor ve dosyada 1000 satır varsa kulp tablanın %10'unu kaplar. Bu formül boş dosyada sıfıra bölmeye yol açar. Hoca, böyle uç durumların analiz sırasında görülüp çözülmesi gerektiğini vurgular. Bu tür türetilmiş değerler ve kısıtlar UML diyagramında gösterilemez; OCL ile yazılır ve ders ilerledikçe OCL'ye giriş yapılacaktır.

Dersin ikinci yarısı genel tasarım kavramlarıdır. Tasarım bilinçli ve amaçlı planlamadır; moda olduğu için mikroservise ya da konteynere geçmek bilinçli tasarım değildir. Mühendislik bilimsel ve matematiksel temellere dayanan üretimdir. Zanaat deneyimden gelen yetenektir (hocanın örneği: yıllar içinde ustalaşan marangozun sağlam, gıcırdamayan sandalyesi). Sanat estetik üretimdir. Hoca, iyi bir ürünün bunların hepsini birleştirdiğini, kullanımı zor ya da kötü görünen ürünlerin zamanla piyasadan silindiğini söyler.

Yazılım tasarımı programlamadan ölçek ve işlevsel olmayan gereksinimlere verilen önemle ayrılır. Programlarken 100, 1000 ya da 10000 satırla bir problem çözülür; Uluslararası Uzay İstasyonu yazılımında 30 milyon satır kod vardır. Tasarımda her zaman bir şeyden ödün verilir. Hocanın hava tahmini örneğinde ızgara üzerinde ilerleyen bir bulut izlenir; performans için dizi tercih edilir. Komşuluk altıgen olabilecekse dizi kullanılamaz ve performanstan ödün verilir.

Mimari tasarım, davranışsal yönleriyle belirlenen sorumlulukları modül ve bileşenlere atamak ve bunların etkileşimini belirlemektir. Detaylı tasarım ise metotların içini, veri yapılarını ve algoritmaları tanımlar. Detaylı tasarım sözde kod, akış diyagramı ya da karar tablosuyla, mimari tasarım ise bu derste UML ile gösterilir. Tasarım yaklaşımları (nesneye yönelik, yapısal, rol tabanlı, gerçek zamanlı, UI/UX) ne olursa olsun yöntem, gösterim ve geçerleme ortaktır. Geçerleme bağımsız bir ekip tarafından, mümkünse erken yapılmalıdır, çünkü geç bulunan hata maliyeti ve takvimi zorlar.

Belgelendirme bölümünde hoca, büyük ve karmaşık sistemlerde belgeye ihtiyacın arttığını, bakım ve değişiklik döneminde iyi belgenin işi kolaylaştırdığını anlatır. Geleneksel belgeler bileşenleri, kontrol akışını, performans kriterlerini ve kaynak kullanımını içerir. Daha ayrıntılı bir çerçeve IEEE 1016 standardıdır: tasarımcı kim, parçalar arası bağımlılıklar, gizli varsayımlar, işlevsel olmayan gereksinimler arası ödünleşimlerin nasıl kararlaştırıldığı (ekrandan, 01:51:10).

Son olarak anahtar kavramlar sayılır. Kavramsal bütünlük, ekibin aynı tasarım anlayışından çıkmasıdır. Az bağımlılık, bir değişikliğin diğer bileşenleri değiştirmeyi gerektirmemesidir. İyi uyum, her bileşenin tek bir amaca hizmet etmesidir ve tekrar kullanımı artırır. Bilgi gizleme (David Parnas), modülün yapabildiklerini soyut bir arayüz arkasında saklamaktır; hoca bunu mikroservislerdeki API Gateway ile örnekler. Soyutlama, bir adımda yalnız o adıma odaklanmaktır; analizde tasarım ayrıntısıyla uğraşılmaz. Estetik, Aquinas'ın bütünlük, ahenk ve saflık üçlüsünün yazılımdaki karşılıklarıdır. Tasarım felsefesinde Descartes analiz ve model kullanımını, Marx kullanıcı merkezli tasarımı, Heidegger araç (CASE, IDE) kullanımını, Wittgenstein ise anlamlı adlandırmayı temsil eder. Hoca bunu "firewall" kelimesinin, ahşap evler arasında yangının sıçramasını önleyen tuğla duvardan geldiğini anlatarak örnekler.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.